Báo cáo thực hành

Chương 2: LẬP TRÌNH PHÁT TRIỂN ỨNG DỤNG DI ĐỘNG

Học phần: Phát triển ứng dụng trên thiết bị di động (KC359), HK2 – NH 2025-2026

Mã nhóm học phần: KC359001

Nhóm 9, danh sách nhóm:

STT	Họ và Tên	MSSV	Mức độ tham gia
1	Trần Nguyên Hiền	B2305223	100%
2	Vưu Quang Nhân	B2305247	100%
3	Lê Thanh Toàn	B2305266	100%
4	Đặng Văn Til	B2305264	100%
5	Trương Hoàng Long	B2305238	100%

Nội dung thực hành:

Câu 2.1 Hãy hiệu chỉnh *StyleSheet* để tách vùng nhập số và kết quả *input_res* thành hai vùng riêng biệt *input* và *result*, *input* chiếm 2.7 phần và *result* chiếm 1.3 phần màn hình. Người đọc ghi nhận *StyleSheet* của *input* và result và kết quả thay đổi trên giao diện vào báo cáo (cần thể hiện được giao diện trước và sau thay đổi).

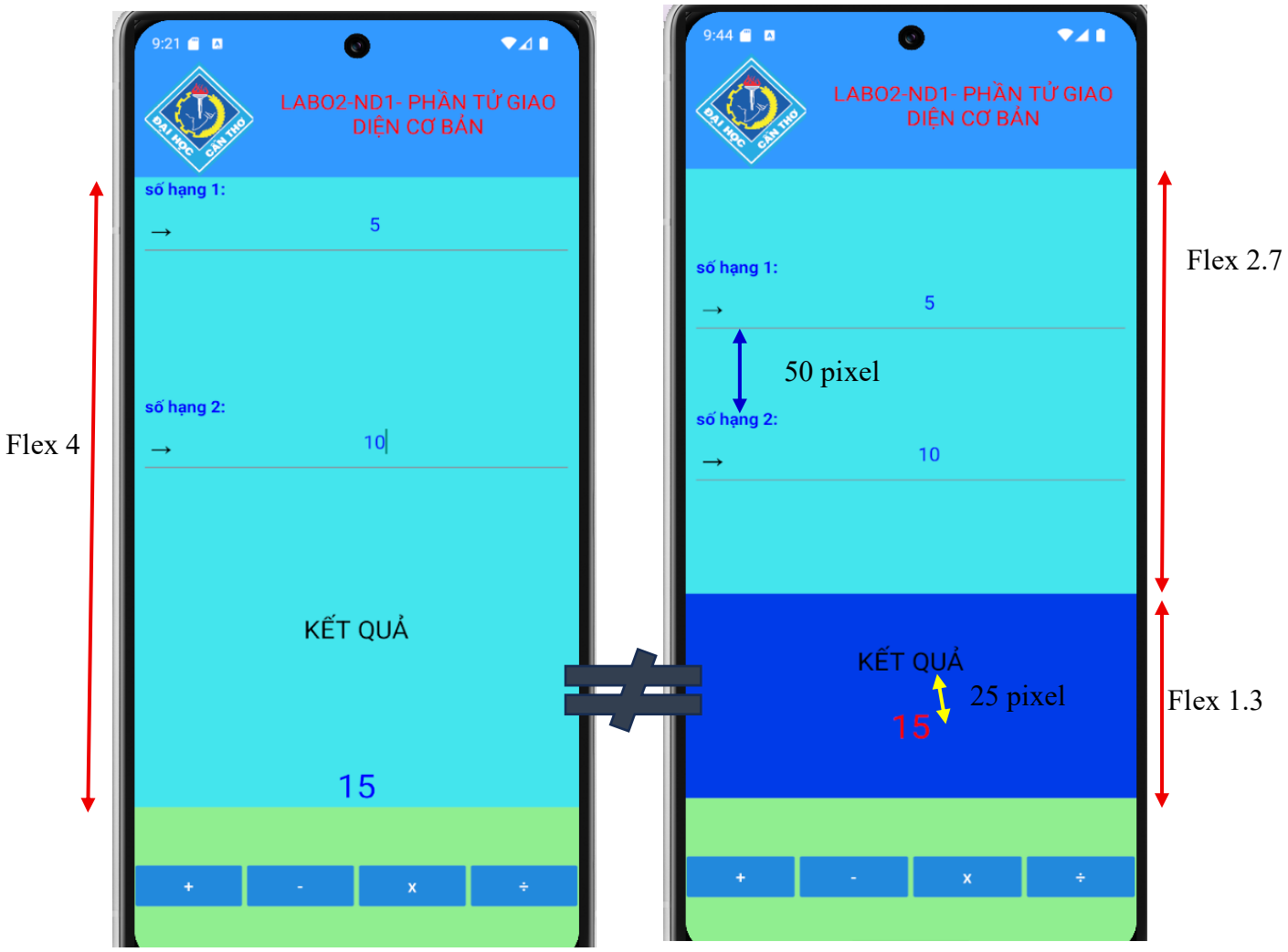
```
input_res: {
  flex: 4,
  // alignItems: 'center',
  // justifyContent: 'space-between',
  // backgroundColor: '#45e5ed',
},

input: {
  flex: 2.7,
  alignItems:'center',
  justifyContent:'center',
  backgroundColor:'#45e5ed',
  gap: 50,
},

result: {
  flex: 1.3,
  alignItems:'center',
```

```
  justifyContent:'center',
  backgroundColor:'#0734e7f8',
  gap: 25,
},
```

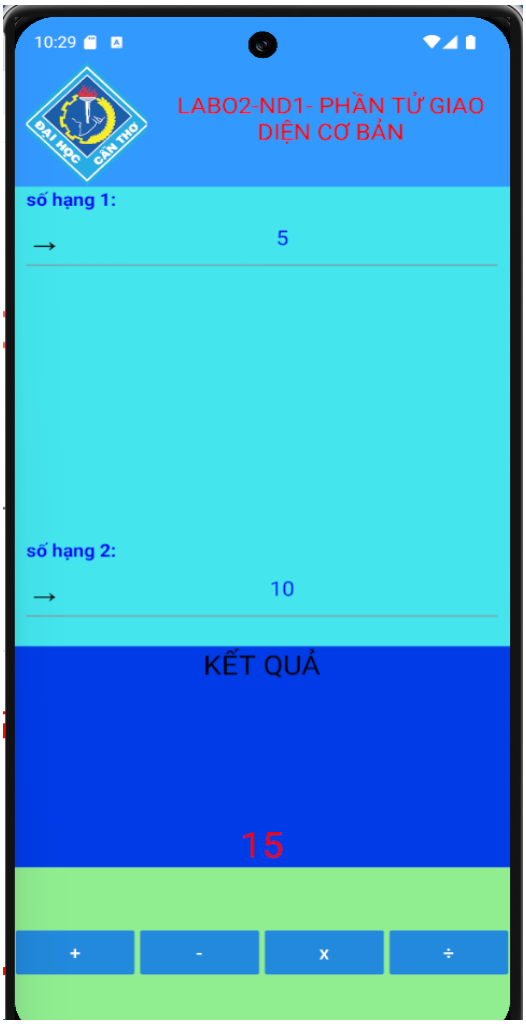
Thay đổi phần input chiếm 2.7 phần kết quả chiếm 1.3 nên flex của của input_res thành 4:

+ Trong phần input đã gôm tất cả phần tử vào trung tâm theo cả chiều ngang và chiều dọc và tạo khoảng cách giữa các phần tử là 50 pixel màu nền là xanh nhạt.

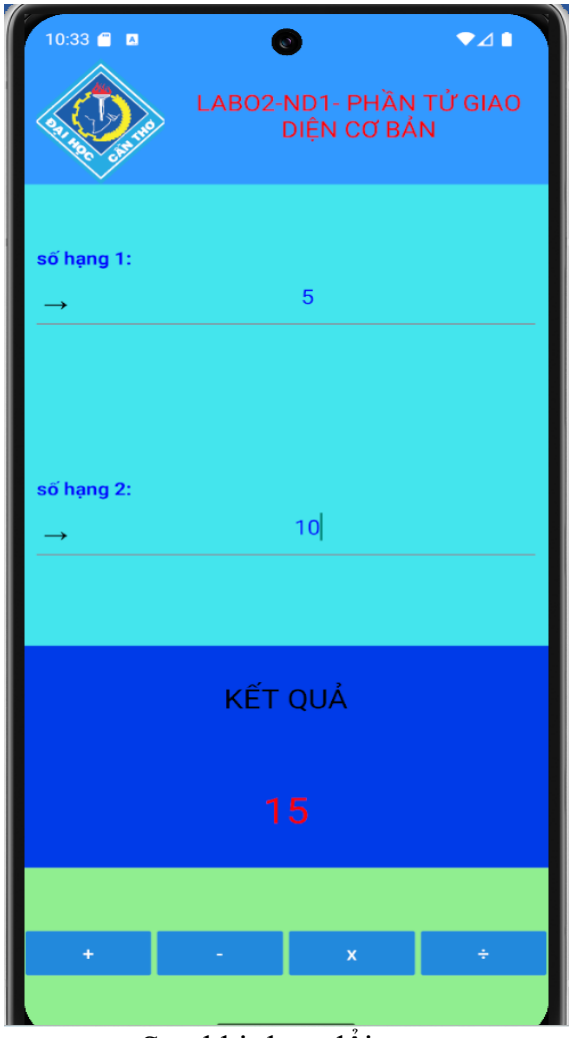
+ Trong phần result cũng gôm tất cả phần tử vào trung tâm theo cả 2 chiều ngang dọc, cũng tạo khoảng cách giữa các phần tử nhưng chỉ 20 pixel màu nền là xanh đậm.



Câu 2.2 Hãy hiệu chỉnh cặp giá trị thuộc tính *justifyContent* của *input* và *result* thành '*space-around'*, quan sát kết quả. Người đọc hãy ghi nhận hình ảnh kết quả trước và sau thay đổi, nhận xét.



Trước Khi thay đổi



Sau khi thay đổi

Trước thay đổi *justifyContent: 'space-between'* làm cho các phần tử đầu và cuối ở sát mép khung.

Sau thay đổi *justifyContent: 'space-around'* làm cho các phần tử đầu và cuối cách mép khung một khoảng trống.

Ta thấy kiểu *'space-between'* dùng để tiết kiệm không gian hoặc các logo sát mép khung,.. còn *'space-around'* dùng cho các biểu tượng hoặc nút bấm trong một khung hình mà không muốn chúng bị sát mép khung. Trong trường hợp của chúng ta có không gian còn nhiều nên tốt nhất chọn kiểu 'space-around' để giao diện trở nên thẩm mỹ hơn.

Câu 2.3 Hãy tìm và thay đổi một số *icon* khác cho 2 *input*. Ghi nhận các lệnh liên quan và màn hình kết quả vào báo cáo.

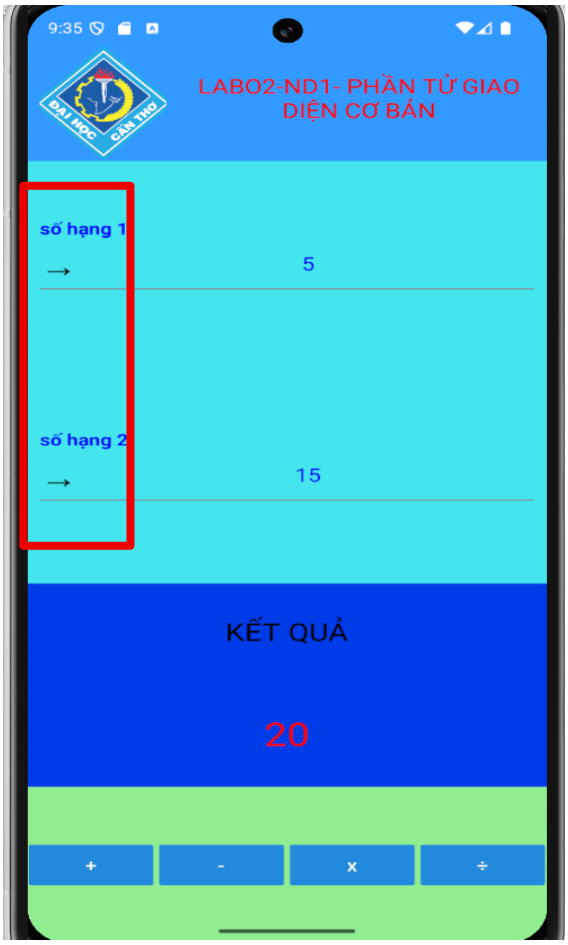
Trước khi thay đổi icon:

```
<Input
  disabledInputStyle={{ background: "#ddd" }}
  label="số hạng 1:"
  leftIcon={<Text style={{ fontSize: 30 }}>x</Text>} // Thay Icon bằng Text để
tránh lỗi font
  placeholder="123"
  color='blue'
  keyboardType="numeric"
  value={sh1}
  onChangeText={(value) => { setSh1(value); }}
  labelStyle={{ color: 'blue' }}
  inputStyle={{ color: 'blue', textAlign: 'center' }}
/>

<Input
  disabledInputStyle={{ background: "#ddd" }}
  label="số hạng 2:"
  leftIcon={<Text style={{ fontSize: 30 }}>x</Text>} // Thay Icon bằng Text
  placeholder="123"
  keyboardType="numeric"
  value={sh2}
  onChangeText={(value) => { setSh2(value); }}
  labelStyle={{ color: 'blue' }}
```

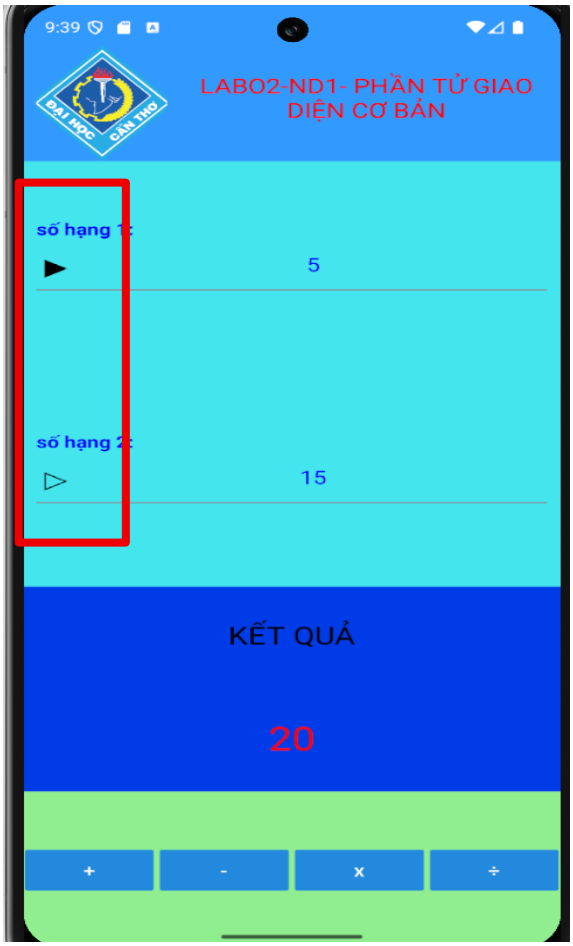
Sau khi thay đổi icon:

```
<View style={styles.input}>
  <Input
    disabledInputStyle={{ background: "#ddd" }}
    label="số hạng 1:"
    leftIcon={<Text style={{ fontSize: 30 }}>▶</Text>} // Thay Icon bằng Text để
tránh lỗi font
    placeholder="123"
    color='blue'
    keyboardType="numeric"
    value={sh1}
    onChangeText={(value) => { setSh1(value); }}
    labelStyle={{ color: 'blue' }}
    inputStyle={{ color: 'blue', textAlign: 'center' }}
  />
```

```
<Input
  disabledInputStyle={{ background: "#ddd" }}
  label="số hạng 2:"
  leftIcon={<Text style={{ fontSize: 30 }}>▶</Text>} // Thay Icon bằng Text
  placeholder="123"
  keyboardType="numeric"
  value={sh2}
  onChangeText={(value) => { setSh2(value); }}
  labelStyle={{ color: 'blue' }}
```



Trước khi thay đổi

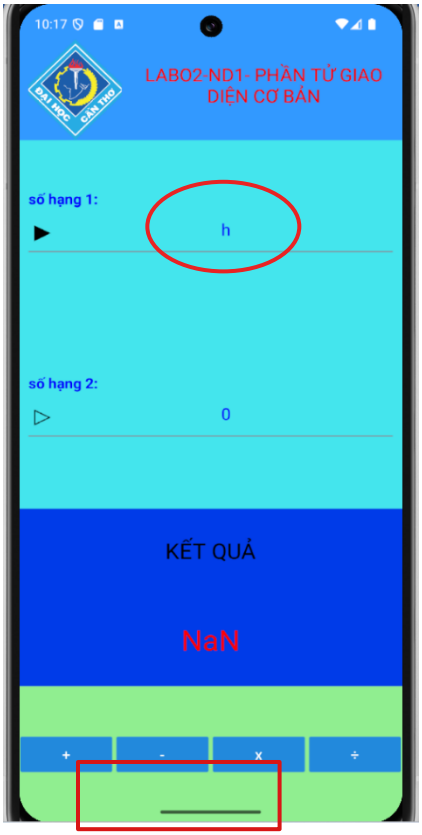


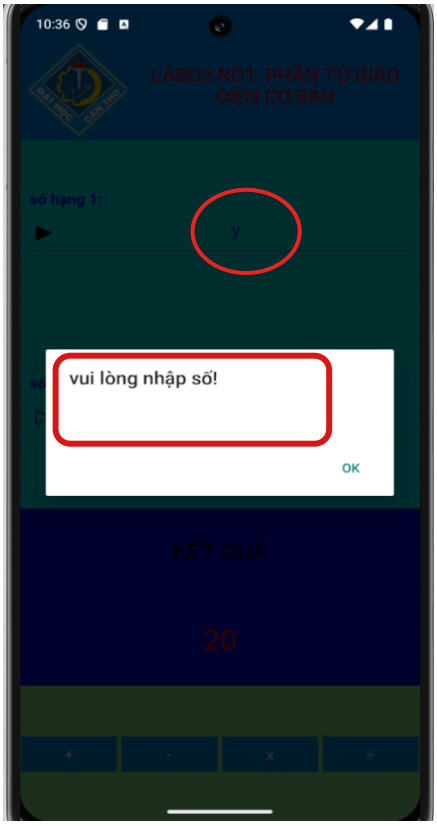
Sau khi thay đổi

Đã thay đổi được từ cả 2 số hạng là icon → đổi sang số hạng thứ nhất là icon ▶ và số hạng thứ 2 là icon ▷.

Câu 2.4 Bổ sung mã lệnh để ứng dụng thông báo cho người sử dụng biết nếu bất kỳ input nào không phải là số khi thực hiện phép toán. Hãy ghi nhận các lệnh liên quan và màn hình kết quả vào báo cáo.

```
const nhan = () => {
  if (!sh1 || !sh2||isNaN(Number(sh1))||isNaN(Number(sh2))) {
    Alert.alert('vui lòng nhập số!');
  } else {
    setKQ(Number(sh1) * Number(sh2));
  }
};

const chia = () => {
  if (!sh1 || !sh2||isNaN(Number(sh1))||isNaN(Number(sh2))) {
    Alert.alert('vui lòng nhập số!');
  }
  else if(Number(sh2)==0){
    Alert.alert('không thể chia cho 0!');
  }
  else {
    setKQ(Number(sh1) / Number(sh2));
  }
};
```





Khi chưa thay đổi, có thể thấy nếu ta đánh chữ vào số hạng thì kết quả sẽ trả về cho ta la NaN, còn sau khi chỉnh sửa thì sẽ hiện lên màn hình dòng chữ "vui lòng nhập số!" nếu ta ấn các phép toán + - x ÷ (cộng trừ nhân chia).

***Câu 2.5**:* Người học hãy tạo ứng dụng giải phương trình bậc nhất hoặc bậc hai với các hệ số nhập từ các *Input* dựa vào ứng dụng Lab02_ND1, chọn loại phương trình sẽ giải bằng *Switch* hoặc *Checkbox.*

Gợi ý: (1) Người học hãy thêm một *Input* để cho phép nhập *sh3*; (2) viết thêm hai hàm giải phương trình bậc nhất và bậc hai với các hệ số tương ứng với các *sh1, sh2 và sh3*; (3) thay *Button* "+" trong vùng *operator* thành "Giải PT", thay hai *Button* còn lại thành *Switch* hoặc *Checkbox*; (4) viết mã lệnh mới cho sự kiện *onPress* của *Button* "Giải PT" để kiểm tra giá trị của *Switch* hoặc *Checkbox* để gọi hàm giải tương ứng.

- Thêm một *Input* để cho phép nhập *sh3*, viết thêm hai hàm giải phương trình bậc nhất và bậc hai:

```
const App = () => {
  const [sh1, setSh1] = useState("");
  const [sh2, setSh2] = useState("");
  const [sh3, setSh3] = useState("");
  const [kq, setKQ] = useState("...");
  const [ptBac2, setPtBac2] = useState(false);

  // Hàm giải PT Bậc 1
  const giaiPtBac1 = (a: number, b: number) => {
    if (a === 0) {
      if (b === 0) return "Vô số nghiệm";
      return "Vô nghiệm";
    }
    const x = -b / a;
    return `x = ${x.toFixed(2)}`;
  };

  // Hàm giải PT Bậc 2
  const giaiPtBac2 = (a: number, b: number, c: number) => {
    if (a === 0) {
      return giaiPtBac1(b, c);
    }
    const delta = b * b - 4 * a * c;
    if (delta < 0) {
      return "Vô nghiệm";
    } else if (delta === 0) {
      const x = -b / (2 * a);
      return `Nghiệm kép: x1 = x2 = ${x.toFixed(2)}`;
```

```
    } else {
      const x1 = (-b + Math.sqrt(delta)) / (2 * a);
      const x2 = (-b - Math.sqrt(delta)) / (2 * a);
      return `x1 = ${x1.toFixed(2)}  |  x2 = ${x2.toFixed(2)}`;
    }
  };
```

- Thay Button thành "Giải PT" và *Switch*:

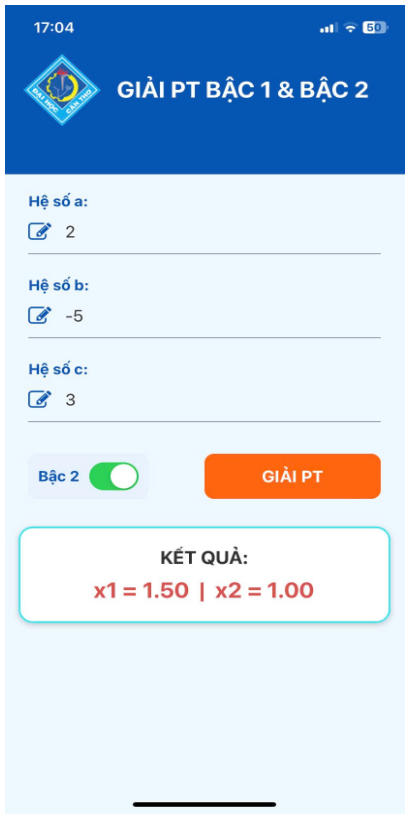
```
{/* Switch và nút GIẢI PT */}
          <View style={styles.actionRow}>
            <View style={styles.switchContainer}>
              <Text style={styles.switchText}>
                {ptBac2 ? "Bậc 2" : "Bậc 1"}
              </Text>
              <Switch
                value={ptBac2}
                onValueChange={(value) => {
                  setPtBac2(value);
                  setKQ("...");
                }}
              />
            </View>

            <Button
              title='GIẢI PT'
              containerStyle={{ width: '50%' }}
              buttonStyle={{ backgroundColor: '#ff6600', borderRadius: 10,
height: 50 }}
              titleStyle={{ fontWeight: 'bold', fontSize: 18 }}
              onPress={() => giaiPT()}
            />
          </View>
```

- Viết mã lệnh mới cho sự kiện *onPress* của *Button* "Giải PT" để kiểm tra giá trị của *Switch*:

```
// Xử lý khi bấm nút GIẢI PT
  const giaiPT = () => {
    if (!sh1 || !sh2) {
      Alert.alert('Lỗi', 'Vui lòng nhập ít nhất hệ số a và b!');
      return;
    }

    const a = Number(sh1);
    const b = Number(sh2);
```

```
    const c = Number(sh3);

    // Kiểm tra xem người dùng có nhập sai định dạng ko
    if (isNaN(a) || isNaN(b) || (ptBac2 && isNaN(c))) {
      Alert.alert('Lỗi', 'Vui lòng chỉ nhập số!');
      return;
    }

    if (giaiPtBac2) {
      if (!sh3) {
        Alert.alert('Lỗi', 'Giải PT Bậc 2 phải nhập đủ hệ số c!');
        return;
      }
      setKQ(giaiPtBac2(a, b, c));
    } else {
      setKQ(giaiPtBac1(a, b));
    }
  };
```

- Hình ảnh kết quả:



Câu 2.6: Viết thêm lệnh vào vùng *header* để vẽ nội dung như Hình 2.9. Hãy ghi nhận các lệnh đã thêm và hình ảnh kết quả vào báo cáo.

- Ghi nhận các lệnh đã thêm:

```
<View style={styles.header}>
        <Image
          source={require('./logo-ctu.png')}
          style={styles.logo}
          resizeMode="contain"
        />
        <View style={styles.textContainer}>
          <Text style={styles.headerText}>
            LAB02-ND2 - PHẦN TỬ GIAO{'\n'}DIỆN CƠ BẢN
          </Text>
        </View>
      </View>
```

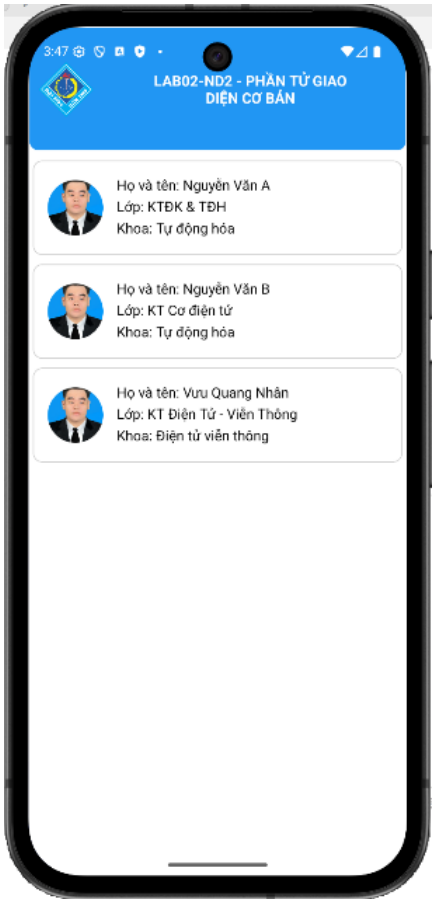
- Hình ảnh kết quả:

Câu 2.7: Người học hãy viết đoạn chương trình để vẽ danh sách vào vùng list. Ghi nhận các lệnh đã thêm và hình ảnh kết quả vào báo cáo.

- Đoạn chương trình để vẽ danh sách vào vùng list:

```
const DATA = [
  {
    id: '1',
    hoten: 'Nguyễn Văn A', // Đổi name -> hoten
    lop: 'KTĐK & TĐH',     // Đổi class -> lop
    khoa: 'Tự động hoá',   // Đổi faculty -> khoa
    path: require('./Quang_Nhan.png'), // Thêm ảnh (Bạn nhớ tạo file này)
  },
  {
    id: '2',
    hoten: 'Nguyễn Văn B',
    lop: 'KT Cơ điện tử',
    khoa: 'Tự động hoá',
    path: require('./Quang_Nhan.png'),
  },
  {
    id: '3',
    hoten: 'Vưu Quang Nhân',
    lop: 'KT Điện Tử - Viễn Thông',
    khoa: 'Điện tử viễn thông',
    path: require('./Quang_Nhan.png'),
  },
];

const renderItem =({ item }: { item: any }) => {
  const backgroundColor = item.id === selectedId ? '#6e3b6e' : "white";
  const color = item.id === selectedId ? 'white' : 'black';

  return (
    <Item
      item={item}
      onPress={() => setSelectedId(item.id)}
      backgroundColor={{ backgroundColor }}
      textColor={{ color }}
    />
  );
};
```

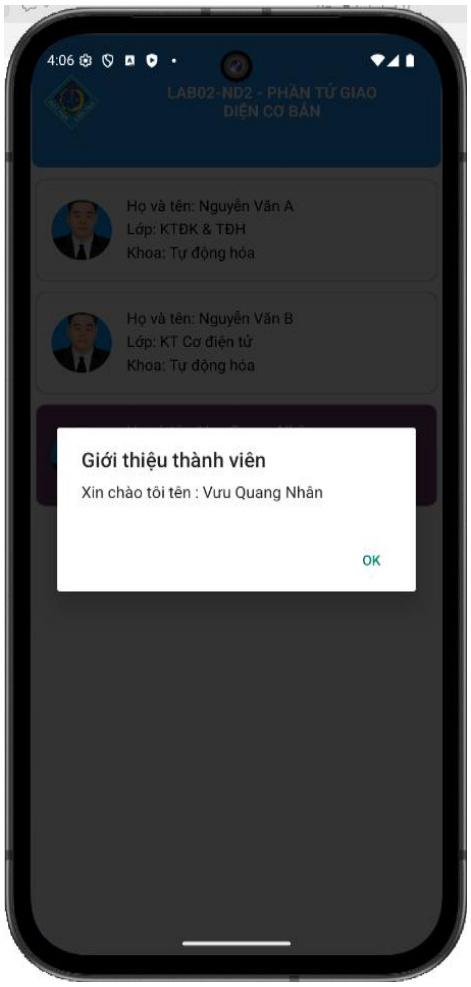
-Hình ảnh hiển thị trên thiết bị Android:



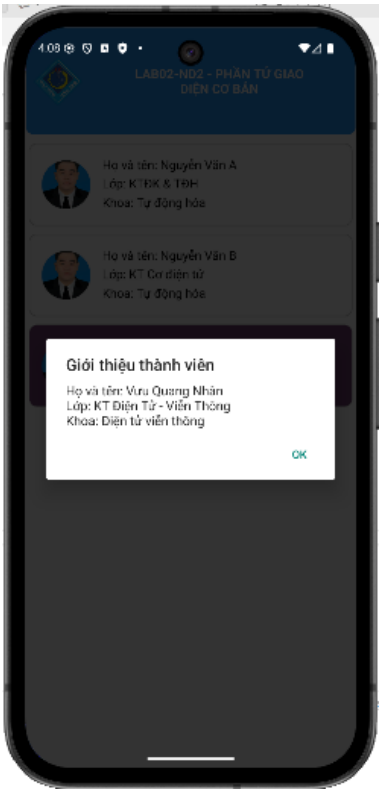
Câu 2.8: Thêm hàm xử lý sự kiện onPress để khi người sử dụng chạm hay click vào hình trong danh sách thì ứng dụng sẽ in ra cửa sổ pop up giới thiệu về thành viên đó, ví dụ như Hình 2.10. Ghi nhận các lệnh đã thêm và hình ảnh kết quả vào báo cáo.

-Đây là hàm hiển thị pop-up giới thiệu thành viên như hình mẫu 2.10 và hình minh họa:

```
      const renderItem = ({ item }: { item: any }) => {
  const backgroundColor = item.id === selectedId ? '#6e3b6e' : "white";
  const color = item.id === selectedId ? 'white' : 'black';

  return (
    <Item
      item={item}
      onPress={() => {
        setSelectedId(item.id);
        Alert.alert(
          'Giới thiệu thành viên',
          `Xin chào tôi tên : ${item.hoten}`
        );
      }}
      backgroundColor={{ backgroundColor }}
      textColor={{ color }}
    />
  );
};
```



-Hoặc ta có thể bổ xung thêm các thông tin khác như thế này .




```
const renderItem = ({ item }: { item: any }) => {
  const backgroundColor = item.id === selectedId ? '#6e3b6e' :
"white";
  const color = item.id === selectedId ? 'white' : 'black';

  return (
    <Item
      item={item}
      onPress={() => {
        setSelectedId(item.id);
        Alert.alert(
          'Giới thiệu thành viên',
          `Họ và tên: ${item.hoten}\nLớp: ${item.lop}\nKhoa:
${item.khoa}`
        );
      }}
      backgroundColor={{ backgroundColor }}
      textColor={{ color }}
    />
  );
};
```

Câu 2.9: Hãy thay đổi thông tin trong danh sách cho đúng với thông tin của các thành viên trong nhóm, bổ sung thêm thông tin giới thiệu về sở thích và sở trường của các thành viên trong nhóm. Ghi nhận các lệnh đã thêm và hình ảnh kết quả vào báo cáo.

-Những đoạn code đã được chỉnh sửa hoặc thêm để cập nhật danh sách cá thành viên trong nhóm .

```
const Item = ({ item, onPress, backgroundColor, textColor }: any) => (
  <TouchableOpacity onPress={onPress} style={[styles.item, backgroundColor]}>
    <Image
      source={item.path}
      style={styles.avatar}
    />
    <View style={styles.textInfo}>
      <Text style={[styles.itemText, textColor]}>MSSV: {item.MSSV}</Text>
      <Text style={[styles.itemText, textColor]}>Họ và tên: {item.hoten}</Text>
      <Text style={[styles.itemText, textColor]}>Lớp: {item.lop}</Text>
      <Text style={[styles.itemText, textColor]}>Khoa: {item.khoa}</Text>
      <Text style={[styles.itemText, textColor]}>Sở thích: {item.sothich}</Text>
```

```
      <Text style={[styles.itemText, textColor]}>Sở Trường:
{item.sotruong}</Text>
    </View>
  </TouchableOpacity>
);
```

(Phía trên là hàm const Item)

```
const App = () => {
  const DATA = [
    {
      MSSV: 'B2350254',
      hoten: 'Trần Nguyễn Hiền',
      lop: 'KTĐT-VT',
      khoa: 'Điện tử - Viễn Thông',
      sothich: 'Hát',
      sotruong: 'PCB mạch và code',
      path: require('./Quang_Nhan.png'),
    },
    {
      MSSV: 'B2350254',
      hoten: 'Vưu Quang Nhân',
      lop: 'KTĐT-VT',
      khoa: 'Điện tử - Viễn Thông',
      sothich: 'Hát',
      sotruong: 'PCB mạch và code',
      path: require('./Quang_Nhan.png'),
    },
    {
      MSSV: 'B2305266',
      hoten: 'Lê Thành',
      lop: 'KTĐT-VT',
      khoa: 'Điện tử - Viễn Thông',
      sothich: 'Hát',
      sotruong: 'PCB mạch và code',
      path: require('./Quang_Nhan.png'),
    },
    {
      MSSV: 'B2350264',
      hoten: 'Đặng Văn Tài',
      lop: 'KTĐT-VT',
      khoa: 'Điện tử - Viễn Thông',
      sothich: 'Hát',
      sotruong: 'PCB mạch và code',
      path: require('./Quang_Nhan.png'),
    },
```

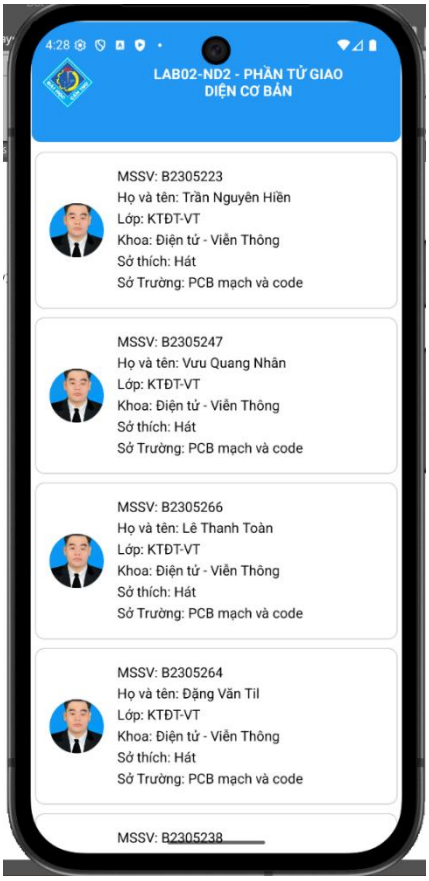
```
    {
      MSSV: 'B2350258',
      hoten: 'Trương Hoàng Long',
      lop: 'KTĐT-VT',
      khoa: 'Điện tử - Viễn Thông',
      sothich: 'Hát',
      sotruong: 'PCB mạch và code',
      path: require('./Quang_Nhan.png'),
    },
  ];
```

(Phía trên hàm const DATA)

```
{/* Phần List View */}
<View style={styles.list_view}>
  <FlatList
    data={DATA}
    renderItem={renderItem}
    keyExtractor={(item) => item.MSSV}
    extraData={selectedId}
    style={{ width: '100%' }}
  />
</View>
</View>
```

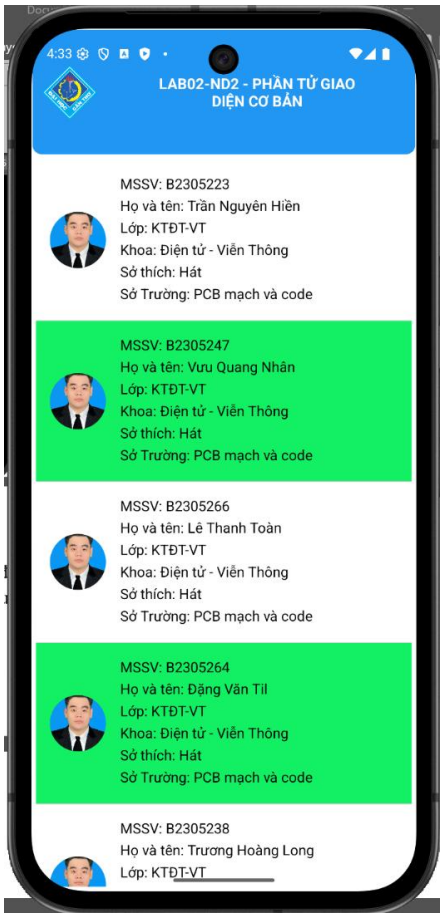
(Phía trên là phần hiển thị danh sách).

-Hình ảnh sau khi cập nhật danh sách các thành viên nhóm:



Câu 2.10: Hiệu chỉnh chương trình để thay đổi luân phiên màu nền của hai phần tử liền kề trong danh sách (thay đổi luân phiên giữa màu nền và một màu khác do người học lựa chọn), ví dụ như Hình 2.11.

```
const renderItem = ({ item, index }: { item: any, index: number }) => {
  const backgroundColor = index % 2 === 0 ? "white" : "#07f45a";

  return (
    <Item
      item={item}
      onPress={() => {
        Alert.alert(
          'Giới thiệu thành viên',
          `Họ và tên: ${item.hoten}\nLớp: ${item.lop}\nKhoa: ${item.khoa}`
        );
      }}
      backgroundColor={{ backgroundColor }}
      textColor={{ color: 'black' }}
    />
  );
};
```



Câu 2.11: (Tự chọn) Hãy thêm chức năng nhập thêm thông tin thành viên cho ứng dụng (sử dụng ảnh trực tuyến, thông tin nhập từ các Input và thêm một nút nhấn để thêm thành viên). Cung cấp mã lệnh chương trình và các hình ảnh kết quả tương ứng vào báo cáo.

```
const [members, setMembers] = useState([
  {
    MSSV: 'B2305223', hoten: 'Trần Nguyễn Hiền', lop: 'KTĐT-VT', khoa: 'Điện tử -
Viễn Thông',
    sothich: 'Hát', sotruong: 'PCB mạch và code ',
    path: require('./Quang_Nhan.png'),
  },
  {
    MSSV: 'B2305247', hoten: 'Vưu Quang Nhân', lop: 'KTĐT-VT', khoa: 'Điện tử -
Viễn Thông',
    sothich: 'Hát', sotruong: 'PCB mạch và code ',
    path: require('./Quang_Nhan.png'),
  },
  {
    MSSV: 'B2305266', hoten: 'Lê Thanh Toàn', lop: 'KTĐT-VT', khoa: 'Điện tử -
Viễn Thông',
    sothich: 'Hát', sotruong: 'PCB mạch và code ',
    path: require('./Quang_Nhan.png'),
  },
  {
    MSSV: 'B2305264', hoten: 'Đặng Văn Tài', lop: 'KTĐT-VT', khoa: 'Điện tử -
Viễn Thông',
    sothich: 'Hát', sotruong: 'PCB mạch và code ',
    path: require('./Quang_Nhan.png'),
  },
  {
    MSSV: 'B2305238', hoten: 'Trương Hoàng Long', lop: 'KTĐT-VT', khoa: 'Điện tử
- Viễn Thông',
    sothich: 'Hát', sotruong: 'PCB mạch và code ',
    path: require('./Quang_Nhan.png'),
  },
]);
```

-Thay đổi const DATA ban đầu thành thế này để có thể tiến hành cập nhật danh sách.

```
const handleAddMember = () => {
  // Kiểm tra
  if(!inputData.mssv || !inputData.hoten) {
    Alert.alert("Thiếu thông tin", "Vui lòng nhập MSSV và Họ tên!");
    return;
  }

  const newMember = {
    MSSV: inputData.mssv,
    hoten: inputData.hoten,
    lop: inputData.lop,
    khoa: inputData.khoa,
    sothich: inputData.sothich,
    sotruong: inputData.sotruong,
    path: inputData.image
  };

  setMembers([...members, newMember]);
  setShowForm(false);

  setInputData({...inputData, mssv: '', hoten: '', lop: '', khoa: '', sothich:
'', sotruong: ''});
  Alert.alert("Thành công", "Đã thêm thành viên mới!");
};
```

- Thêm phần form nhập thành viên mới và các bước kiểm tra trong form.

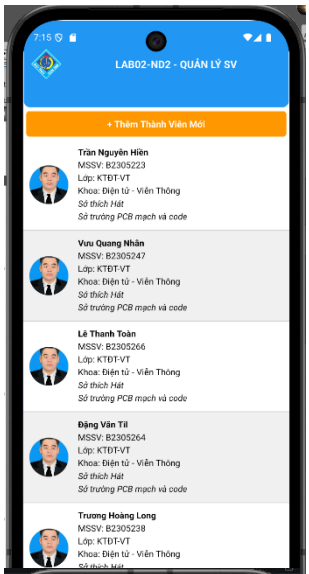
```
<View>
  <TouchableOpacity style={styles.toggleButton} onPress={() =>
setShowForm(!showForm)}>
    <Text style={styles.buttonText}>
      {showForm ? "Đóng Form Nhập Liệu" : "+ Thêm Thành Viên Mới"}
    </Text>
  </TouchableOpacity>

  {showForm && (
    <ScrollView style={styles.formContainer}>
      <TextInput
        style={styles.input}
        placeholder="Link Ảnh Online (URL)"
        value={inputData.image}
```

```
      onChangeText={(text) => setInputData({ ...inputData, image: text })}
    />

    <View style={{ flexDirection: 'row', justifyContent: 'space-between' }}>
      <TextInput
        style={[styles.input, { flex: 1, marginRight: 5 }]}
        placeholder="MSSV"
        value={inputData.mssv}
        onChangeText={(text) => setInputData({ ...inputData, mssv: text })}
      />
      <TextInput
        style={[styles.input, { flex: 1 }]}
        placeholder="Lớp"
        value={inputData.lop}
        onChangeText={(text) => setInputData({ ...inputData, lop: text })}
      />
    </View>

    <TextInput
      style={styles.input}
      placeholder="Họ và Tên"
      value={inputData.hoten}
      onChangeText={(text) => setInputData({ ...inputData, hoten: text })}
    />

    <TextInput
      style={styles.input}
      placeholder="Khoa"
      value={inputData.khoa}
      onChangeText={(text) => setInputData({ ...inputData, khoa: text })}
    />

    <TextInput
      style={styles.input}
      placeholder="Sở thích"
      value={inputData.sothich}
      onChangeText={(text) => setInputData({ ...inputData, sothich: text })}
    />

    <TextInput
      style={styles.input}
      placeholder="Sở trường"
      value={inputData.sotruong}
      onChangeText={(text) => setInputData({ ...inputData, sotruong: text })}
    />
```

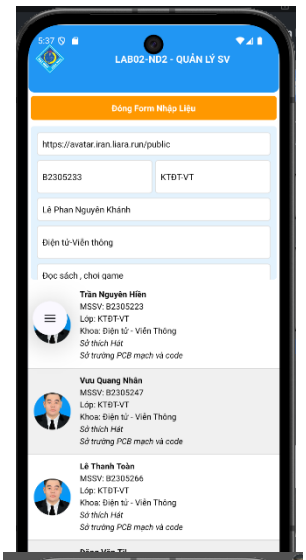
```
      <TouchableOpacity style={styles.addButton} onPress={handleAddMember}>
        <Text style={styles.buttonText}>LƯU THÀNH VIÊN</Text>
      </TouchableOpacity>
    </ScrollView>
  )}
</View>
```

-Tạo thêm các nút nhấn phù hợp với chức năng.

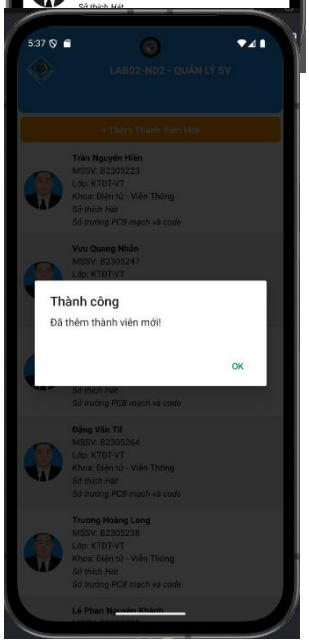
-Hình ảnh thực tế :



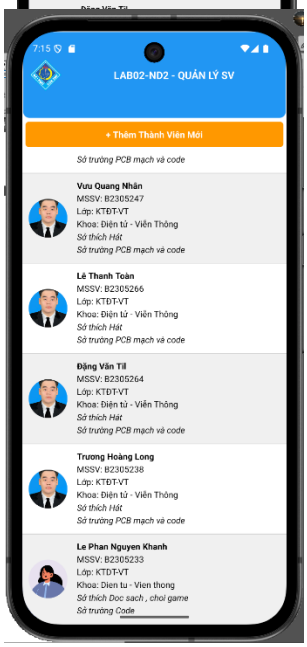
-Nút "Thêm Thành Viên Mới



-Hình dáng form thêm thành viên



-Sau khi thêm thành công



-Danh sách được cập nhật sau khi thêm

Câu 2.12: Dựa vào chương trình của ứng dụng trong phần 2.2, người học hãy tích hợp tính năng tính toán vào màn hình Calculation. Ghi nhận các lệnh đã thêm và hình ảnh kết quả vào báo cáo.

- Các lệnh đã được thêm vào chương trình để thêm chức năng tính toán:

```
import React, { useState } from 'react';
```

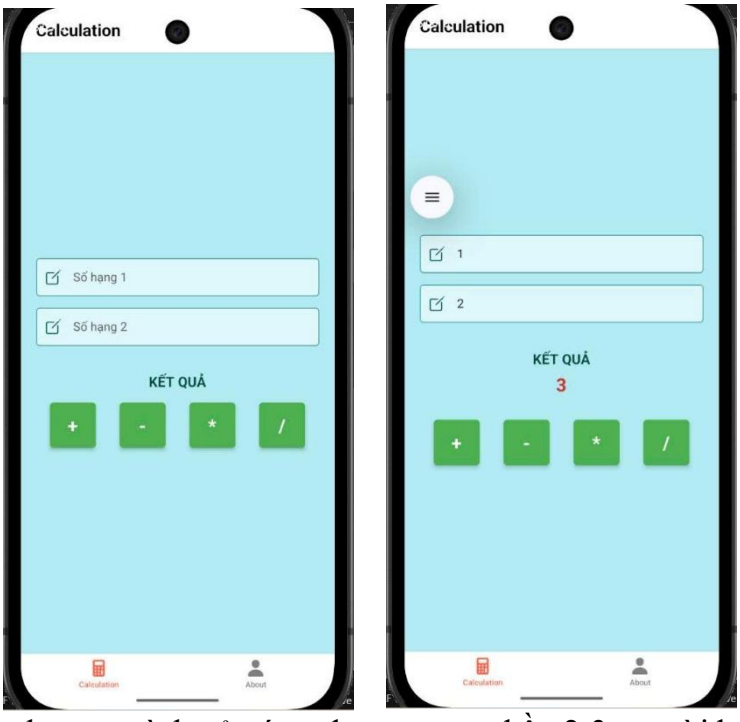
```
const CalculationScreen = () => {

  const [number1, setNumber1] = useState('');
  const [number2, setNumber2] = useState('');
  const [result, setResult] = useState('');

  const calculate = (operator: string) => {
    const num1 = parseFloat(number1);
    const num2 = parseFloat(number2);

    if (isNaN(num1) || isNaN(num2)) {
      setResult('Vui lòng nhập số');
      return;
    }

    let res;
    switch (operator) {
      case '+': res = num1 + num2; break;
      case '-': res = num1 - num2; break;
      case '*': res = num1 * num2; break;
      case '/': res = num2 !== 0 ? num1 / num2 : 'Lỗi chia 0'; break;
      default: res = '';
    }

    setResult(res.toString());
  };

};
```

-Lưu dữ liệu nhập và kết quả cũng như hàm sử lý tính toán.

-Giao diện của nhập số và các phép toán và hiển thị kết quả.

```
<View style={styles.bodyContainer}>

  <View style={styles.inputWrapper}>
    <Icon name="create-outline" size={24} color="#006064"
style={styles.inputIcon} />
    <TextInput
      style={styles.input}
      placeholder="Số hạng 1"
      keyboardType="numeric"
      value={number1}
      onChangeText={setNumber1}
      placeholderTextColor="#555"
    />
  </View>

  <View style={styles.inputWrapper}>
    <Icon name="create-outline" size={24} color="#006064"
style={styles.inputIcon} />
    <TextInput
      style={styles.input}
      placeholder="Số hạng 2"
      keyboardType="numeric"
      value={number2}
      onChangeText={setNumber2}
      placeholderTextColor="#555"
    />
  </View>

  <Text style={styles.resultLabel}>KẾT QUẢ</Text>
  {result !== '' && <Text style={styles.resultValue}>{result}</Text>}

  <View style={styles.buttonRow}>
    {['+', '-', '*', '/'].map((op) => (
      <TouchableOpacity
        key={op}
        style={styles.opButton}
        onPress={() => calculate(op)}
      >
        <Text style={styles.opText}>{op}</Text>
      </TouchableOpacity>
    ))}
  </View>

</View>
```

- Hình ảnh được hiển thị trên thiết bị Android:



Câu 2.13: Dựa vào chương trình của ứng dụng trong phần 2.3, người học hãy thiết kế giao diện cho màn hình About giới thiệu về các thành viên trong nhóm thực tập bao gồm các thông tin như mã số sinh viên, họ và tên, nội dung phụ trách, phần trăm tham gia thực tập. Ghi nhận các lệnh đã thêm và hình ảnh kết quả vào báo cáo.

- Các lệnh đã thêm các nội dung bao gồm các thông tin như mã số sinh viên, họ và tên, nội dung phụ trách, phần trăm tham gia thực tập của mỗi thành viên trong nhóm:

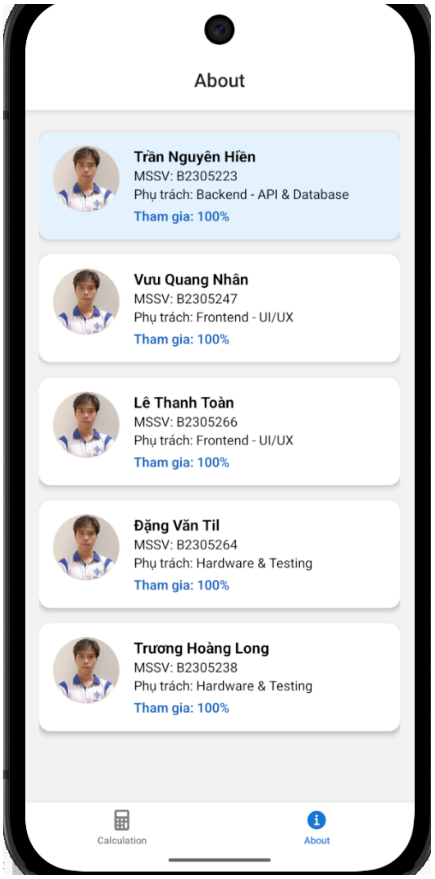
```
const DATA: Member[] = [
  {
    id: '1',
    mssv: 'B2305223',
    hoten: 'Trần Nguyên Hiền',
    phutrach: 'Backend - API & Database',
    phantram: 100,
    avatar: require('./assets/Image_Person.jpg'),
  },
  {
    id: '2',
    mssv: 'B2305247',
    hoten: 'Vưu Quang Nhân',
    phutrach: 'Frontend - UI/UX',
    phantram: 100,
    avatar: require('./assets/Image_Person.jpg'),
  },
```

```
  {
    id: '3',
    mssv: 'B2305266',
    hoten: 'Lê Thanh Toàn',
    phutrach: 'Frontend - UI/UX',
    phantram: 100,
    avatar: require('./assets/Image_Person.jpg'),
  },
  {
    id: '4',
    mssv: 'B2305264',
    hoten: 'Đặng Văn Til',
    phutrach: 'Hardware & Testing',
    phantram: 100,
    avatar: require('./assets/Image_Person.jpg'),
  },
  {
    id: '5',
    mssv: 'B2305238',
    hoten: 'Trương Hoàng Long',
    phutrach: 'Hardware & Testing',
    phantram: 100,
    avatar: require('./assets/Image_Person.jpg'),
  },
];
```

```
interface ItemProps {
  item: Member;
  selected: boolean;
  onPress: () => void;
}

const MemberItem = memo(({ item, selected, onPress }: ItemProps) => {
  return (
    <TouchableOpacity
      style={[
        styles.card,
        { backgroundColor: selected ? '#E3F2FD' : '#FFFFFF' },
      ]}
      onPress={onPress}
      activeOpacity={0.8}
    >
      <Image source={item.avatar} style={styles.avatar} />
```

```
      <View style={styles.infoContainer}>
        <Text style={styles.name}>{item.hoten}</Text>
        <Text style={styles.text}>MSSV: {item.mssv}</Text>
        <Text style={styles.text}>Phụ trách: {item.phutrach}</Text>
        <Text style={styles.percent}>
          Tham gia: {item.phantram}%
        </Text>
      </View>
    </TouchableOpacity>
  );
});
```

- Giao diện ở tab About:



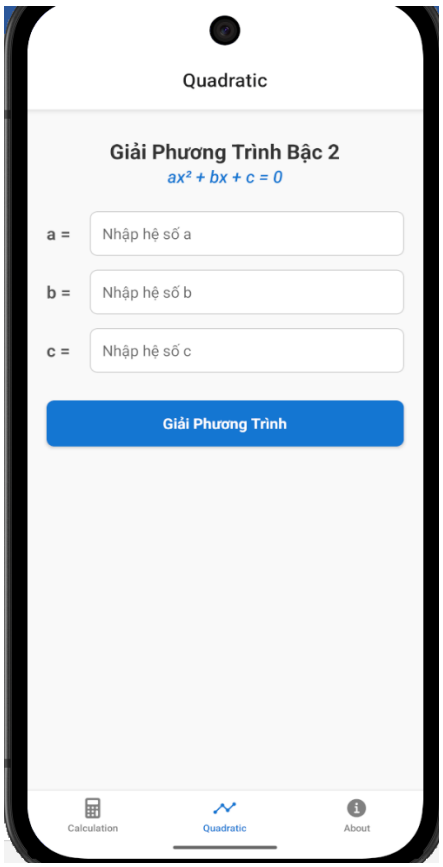
Câu 2.14: Hãy tích hợp thêm màn hình thực hiện chức năng giải phương trình bậc 2 vào ứng dụng. Ghi nhận các lệnh đã thêm và hình ảnh kết quả vào báo cáo.

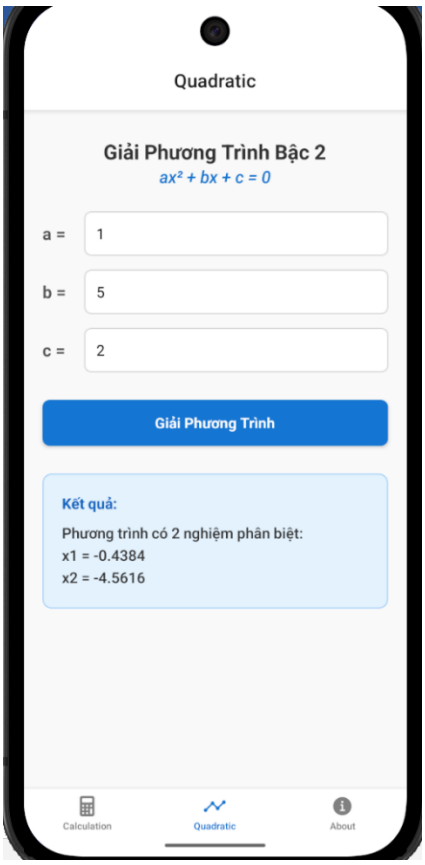
```
const QuadraticScreen = () => {
  const [a, setA] = useState('');
  const [b, setB] = useState('');
  const [c, setC] = useState('');
  const [result, setResult] = useState<string | null>(null);

  const solveQuadratic = () => {
```

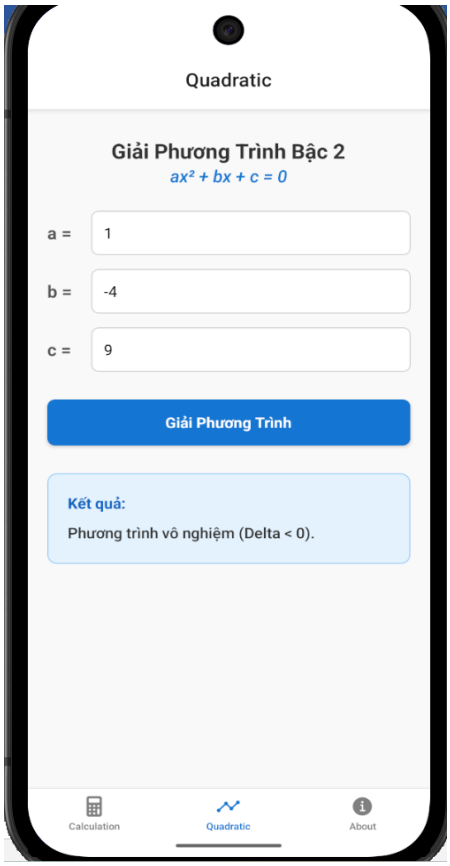
```
    const valA = parseFloat(a);
    const valB = parseFloat(b);
    const valC = parseFloat(c);

    if (isNaN(valA) || isNaN(valB) || isNaN(valC)) {
      setResult('Vui lòng nhập đầy đủ các hệ số a, b, c hợp lệ.');
      return;
    }

    if (valA === 0) {
      if (valB === 0) {
        setResult(valC === 0 ? 'Phương trình có vô số nghiệm.' : 'Phương trình vô
nghiệm.');
      } else {
        const x = -valC / valB;
        setResult(`Hệ số a=0, phương trình bậc 1 có một nghiệm:\nx = ${x}`);
      }
      return;
    }

    const delta = valB * valB - 4 * valA * valC;

    if (delta < 0) {
      setResult('Phương trình vô nghiệm (Delta < 0).');
    } else if (delta === 0) {
      const x = -valB / (2 * valA);
      setResult(`Phương trình có nghiệm kép:\nx1 = x2 = ${x}`);
    } else {
      const x1 = (-valB + Math.sqrt(delta)) / (2 * valA);
      const x2 = (-valB - Math.sqrt(delta)) / (2 * valA);
      setResult(`Phương trình có 2 nghiệm phân biệt:\nx1 = ${x1.toFixed(4)}\nx2 =
${x2.toFixed(4)}`);
    }
  };

  return (
    <SafeAreaView style={styles.quadContainer}>
      <Text style={styles.quadTitle}>Giải Phương Trình Bậc 2</Text>
      <Text style={styles.quadFormula}>ax² + bx + c = 0</Text>

      <View style={styles.inputGroup}>
        <Text style={styles.quadLabel}>a =</Text>
        <TextInput
          style={styles.quadInput}
          keyboardType="numeric"
```

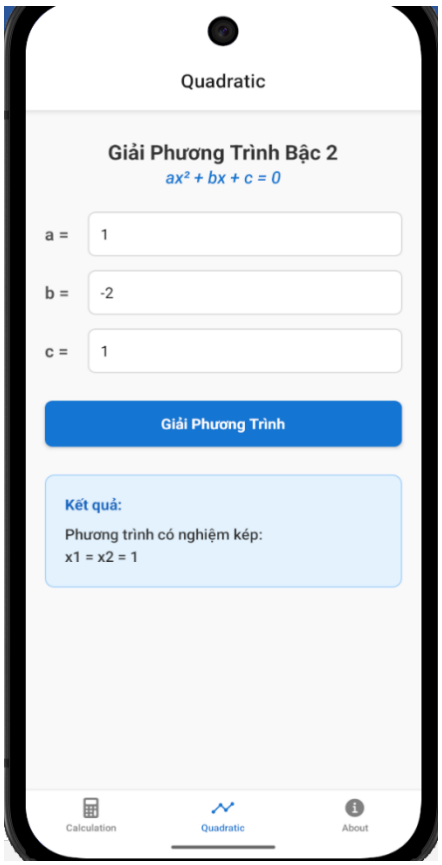
```
          placeholder="Nhập hệ số a"
          value={a}
          onChangeText={setA}
        />
      </View>

      <View style={styles.inputGroup}>
        <Text style={styles.quadLabel}>b =</Text>
        <TextInput
          style={styles.quadInput}
          keyboardType="numeric"
          placeholder="Nhập hệ số b"
          value={b}
          onChangeText={setB}
        />
      </View>

      <View style={styles.inputGroup}>
        <Text style={styles.quadLabel}>c =</Text>
        <TextInput
          style={styles.quadInput}
          keyboardType="numeric"
          placeholder="Nhập hệ số c"
          value={c}
          onChangeText={setC}
        />
      </View>

      <TouchableOpacity style={styles.solveButton} onPress={solveQuadratic}>
        <Text style={styles.solveButtonText}>Giải Phương Trình</Text>
      </TouchableOpacity>

      {result !== null && (
        <View style={styles.resultBox}>
          <Text style={styles.resultLabel}>Kết quả:</Text>
          <Text style={styles.resultTextBox}>{result}</Text>
        </View>
      )}
    </SafeAreaView>
  );
};
```

- Giao diện ở tab Quadratic:


Quadratic
Giải Phương Trình Bậc 2
ax² + bx + c = 0
a =
Nhập hệ số a
b =
Nhập hệ số b
c =
Nhập hệ số c
Giải Phương Trình
Calculation
Quadratic
About


Quadratic
Giải Phương Trình Bậc 2
ax² + bx + c = 0
a =
1
b =
5
c =
2
Giải Phương Trình
Kết quả:
Phương trình có 2 nghiệm phân biệt:
x1 = -0.4384
x2 = -4.5616
Calculation
Quadratic
About


Quadratic
Giải Phương Trình Bậc 2
ax² + bx + c = 0
a =
1
b =
-4
c =
9
Giải Phương Trình
Kết quả:
Phương trình vô nghiệm (Delta < 0).
Calculation
Quadratic
About


Quadratic
Giải Phương Trình Bậc 2
ax² + bx + c = 0
a =
1
b =
-2
c =
1
Giải Phương Trình
Kết quả:
Phương trình có nghiệm kép:
x1 = x2 = 1
Calculation
Quadratic
About

Câu 2.15: (Tự chọn) Người học hãy tìm hiểu thư viện react-native-chart-kit, sau đó tích hợp thêm chức năng vẽ lại đồ thị của phương trình bậc 2 ở Câu 2.14. Ghi nhận các lệnh đã thêm và hình ảnh kết quả vào báo cáo.

```
const QuadraticScreen = () => {
  const [a, setA] = useState('');
  const [b, setB] = useState('');
  const [c, setC] = useState('');
  const [result, setResult] = useState<string | null>(null);
  const [chartData, setChartData] = useState<{
    labels: string[];
    datasets: { data: number[]; color?: (opacity: number) => string;
strokeWidth?: number }[];
  } | null>(null);

  const solveQuadratic = () => {
    const valA = parseFloat(a);
    const valB = parseFloat(b);
    const valC = parseFloat(c);

    if (isNaN(valA) || isNaN(valB) || isNaN(valC)) {
      setResult('Vui lòng nhập đầy đủ các hệ số a, b, c hợp lệ.');
      setChartData(null);
      return;
    }

    if (valA === 0) {
      if (valB === 0) {
        setResult(valC === 0 ? 'Phương trình có vô số nghiệm.' : 'Phương trình vô
nghiệm.');
      } else {
        const x = -valC / valB;
        setResult(`Hệ số a=0, phương trình bậc 1 có một nghiệm:\nx = ${x}`);
      }
      setChartData(null);
      return;
    }

    const delta = valB * valB - 4 * valA * valC;

    let resultText = '';
    if (delta < 0) {
      resultText = 'Phương trình vô nghiệm (Delta < 0).';
    } else if (delta === 0) {
      const x = -valB / (2 * valA);
```

```
      resultText = `Phương trình có nghiệm kép:\nx1 = x2 = ${x}`;
    } else {
      const x1 = (-valB + Math.sqrt(delta)) / (2 * valA);
      const x2 = (-valB - Math.sqrt(delta)) / (2 * valA);
      resultText = `Phương trình có 2 nghiệm phân biệt:\nx1 =
${x1.toFixed(4)}\nx2 = ${x2.toFixed(4)}`;
    }
    setResult(resultText);

    plotGraph(valA, valB, valC);
  };

  const plotGraph = (aNum: number, bNum: number, cNum: number) => {
    const vertexX = -bNum / (2 * aNum);

    const delta = bNum * bNum - 4 * aNum * cNum;
    let span = 5;

    if (delta >= 0) {
      const x1 = (-bNum - Math.sqrt(delta)) / (2 * aNum);
      const padding = 2;
      span = Math.abs(vertexX - x1) + padding;
    }
    span = Math.min(Math.max(span, 3), 20);

    const xMin = vertexX - span;
    const xMax = vertexX + span;
    const numPoints = 40;
    const step = (xMax - xMin) / numPoints;
    const labels: string[] = [];
    const dataPoints: number[] = [];

    for (let i = 0; i <= numPoints; i++) {
      const x = xMin + i * step;
      const y = aNum * x * x + bNum * x + cNum;

      if (i % (numPoints / 4) === 0) {
        labels.push(x.toFixed(1));
      } else {
        labels.push('');
      }

      dataPoints.push(y);
    }
```

```
    setChartData({
      labels: labels,
      datasets: [
        {
          data: dataPoints,
          color: (opacity = 1) => `rgba(25, 118, 210, ${opacity})`,
          strokeWidth: 2,
        },
      ],
    });
  };

  return (
    <SafeAreaView style={styles.quadContainer}>
      <ScrollView showsVerticalScrollIndicator={false} contentContainerStyle={{
paddingBottom: 40 }}>
        <Text style={styles.quadTitle}>Giải Phương Trình Bậc 2</Text>
        <Text style={styles.quadFormula}>ax² + bx + c = 0</Text>

        <View style={styles.inputGroup}>
          <Text style={styles.quadLabel}>a =</Text>
          <TextInput
            style={styles.quadInput}
            keyboardType="numeric"
            placeholder="Nhập hệ số a"
            value={a}
            onChangeText={setA}
          />
        </View>

        <View style={styles.inputGroup}>
          <Text style={styles.quadLabel}>b =</Text>
          <TextInput
            style={styles.quadInput}
            keyboardType="numeric"
            placeholder="Nhập hệ số b"
            value={b}
            onChangeText={setB}
          />
        </View>

        <View style={styles.inputGroup}>
          <Text style={styles.quadLabel}>c =</Text>
          <TextInput
            style={styles.quadInput}
```

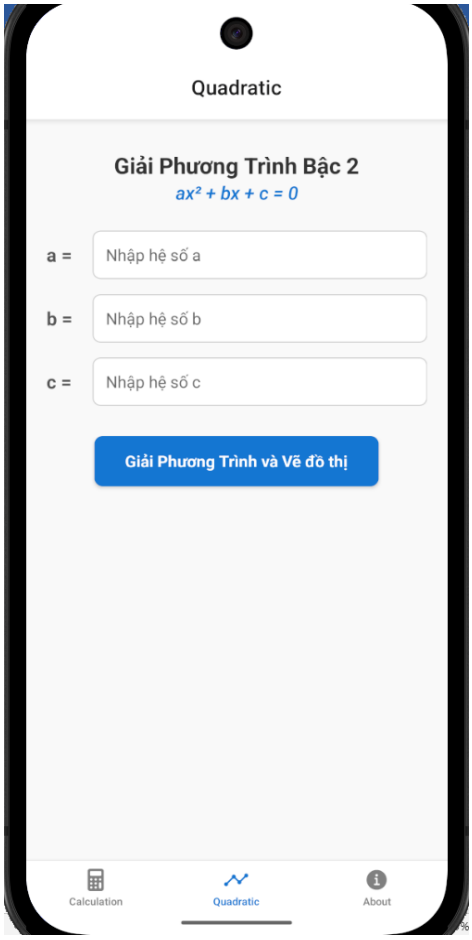
```
            keyboardType="numeric"
            placeholder="Nhập hệ số c"
            value={c}
            onChangeText={setC}
          />
        </View>

        <View style={styles.quadButtonRow}>
          <TouchableOpacity style={styles.solveButton} onPress={solveQuadratic}>
            <Text style={styles.solveButtonText}>Giải Phương Trình và Vẽ đồ
thị</Text>
          </TouchableOpacity>
        </View>

        {result !== null && (
          <View style={styles.resultBox}>
            <Text style={styles.resultLabel}>Kết quả:</Text>
            <Text style={styles.resultTextBox}>{result}</Text>
          </View>
        )}

        {chartData && (
          <View style={styles.chartContainer}>
            <Text style={styles.chartTitle}>Đồ thị hàm số y = {a}x² + {b}x +
{c}</Text>
            <LineChart
              data={chartData}
              width={screenWidth - 40}
              height={220}
              withDots={false}
              chartConfig={{
                backgroundColor: '#ffffff',
                backgroundGradientFrom: '#ffffff',
                backgroundGradientTo: '#ffffff',
                decimalPlaces: 1,
                color: (opacity = 1) => `rgba(25, 118, 210, ${opacity})`,
                labelColor: (opacity = 1) => `rgba(100, 100, 100, ${opacity})`,
                style: {
                  borderRadius: 16,
                },
                propsForDots: {
                  r: '0',
                  strokeWidth: '0',
                },
              }}
```

```
              style={{
                marginVertical: 8,
                borderRadius: 16,
              }}
            />
          </View>
        )}
      </ScrollView>
    </SafeAreaView>
  );
};
```

- Giao diện sau khi tích hợp thêm biểu đồ:

Quadratic
c =
2
Giải Phương Trình và Vẽ đồ thị
Kết quả:
Phương trình có 2 nghiệm phân biệt:
x1 = 2.0000
x2 = 1.0000
Đồ thị hàm số y = 1x² + -3x + 2
8.8
6.5
4.3
2.0
-0.3
-1.5
0.0
1.5
3.0
4.5
Calculation
Quadratic
About

Quadratic
c =
1
Giải Phương Trình và Vẽ đồ thị
Kết quả:
Phương trình có nghiệm kép:
x1 = x2 = 1
Đồ thị hàm số y = 1x² + -2x + 1
9.0
6.8
4.5
2.3
0.0
-2.0
-0.5
1.0
2.5
4.0
Calculation
Quadratic
About


Quadratic
Giải Phương Trình Bậc 2
ax² + bx + c = 0
a =
Nhập hệ số a
b =
Nhập hệ số b
c =
Nhập hệ số c
Giải Phương Trình và Vẽ đồ thị
Calculation
Quadratic
About

Quadratic
c =
9
Giải Phương Trình và Vẽ đồ thị
Kết quả:
Phương trình có 2 nghiệm phân biệt:
x1 = 3.0000
x2 = 1.0000
Đồ thị hàm số y = 3x² + -12x + 9
24.0
17.3
10.5
3.8
-3.0
-1.0
0.5
2.0
3.5
5.0
Calculation
Quadratic
About